ChatGPT

Executive Summary

Ollama's **Model Context Protocol (MCP)** integration lets local LLMs call out to external tools (e.g. unit converters, parsers). We survey Ollama's architecture (local/Cloud modes, supported model formats, and plugin APIs) and security aspects, then review **scientific notation standards** (SI units, E-notation, LaTeX, IUPAC names, InChI/SMILES, FASTA sequences, etc.) and how to represent them in code or ontology form. We analyze the **Qwen3-8B** model (tokenization, context length, adapters/LoRA/PEFT support, "thinking" mode, function-calling APIs) from its official docs and papers. Based on this, we propose a concrete **integration design** for embedding scientific tools into an Ollama MCP server, with JSON schema interfaces and adapter layers. We include example tool implementations (e.g. unit conversion using Pint, scientific notation parser, LaTeX↔text converter, chemical ID resolver, sequence formatter), along with error handling, test strategies, and metrics for correctness. We discuss security (sandboxing LLM/tool execution) and performance (quantization, caching). The report includes code snippets, API specs, a phased integration plan with milestones (and effort/risk), comparison tables of options (formats, libraries, tradeoffs), and mermaid diagrams of the system architecture and data flow. All information is cited from official documentation, standards bodies, and primary sources whenever possible.

1. Ollama MCP Overview

Ollama provides a local (or hosted) LLM runtime with plugin support via **MCP servers**. An **MCP (Model Context Protocol)** server is essentially a thin "tool-hosting" service that exposes helper functions to the LLM in a standard JSON/RPC-like format. For example, *rawveg/ollama-mcp* is an open-source MCP server exposing the entire Ollama CLI/SDK as MCP tools【1†L274-L281】. It uses a hot-swap architecture where each tool is a TypeScript file: simply dropping a new `src/tools/*.ts` adds a new tool with zero server changes【31†L586-L591】. This auto-discovery allows developers to plug in custom tools (unit converters, parsers, etc.) easily.

Architecture & Deployment: An Ollama MCP setup typically runs Ollama's LLM engine (locally or in a container) alongside an MCP server. Ollama models can be local binaries or accessed via the Ollama Cloud API. The environment variable `OLLAMA_HOST` selects local vs. remote service【12†L442-L451】. The rawveg MCP server for Ollama is a Node.js app (usable via `uvx mcp-ollama` or as an npm package), or one can roll a Python/other MCP server. Deployment can be on a developer's workstation, on-premise servers, or in a cloud sandbox. Community projects demonstrate safe deployments using Docker or microVMs: for example, a Docker Compose setup isolates the Ollama runtime with no internet access while a separate updater container fetches models【56†L238-L247】【55†L1782-L1786】. HopX even offers a sandbox platform where each Ollama instance runs in its own hardware-level microVM for strong isolation【55†L1782-L1786】.

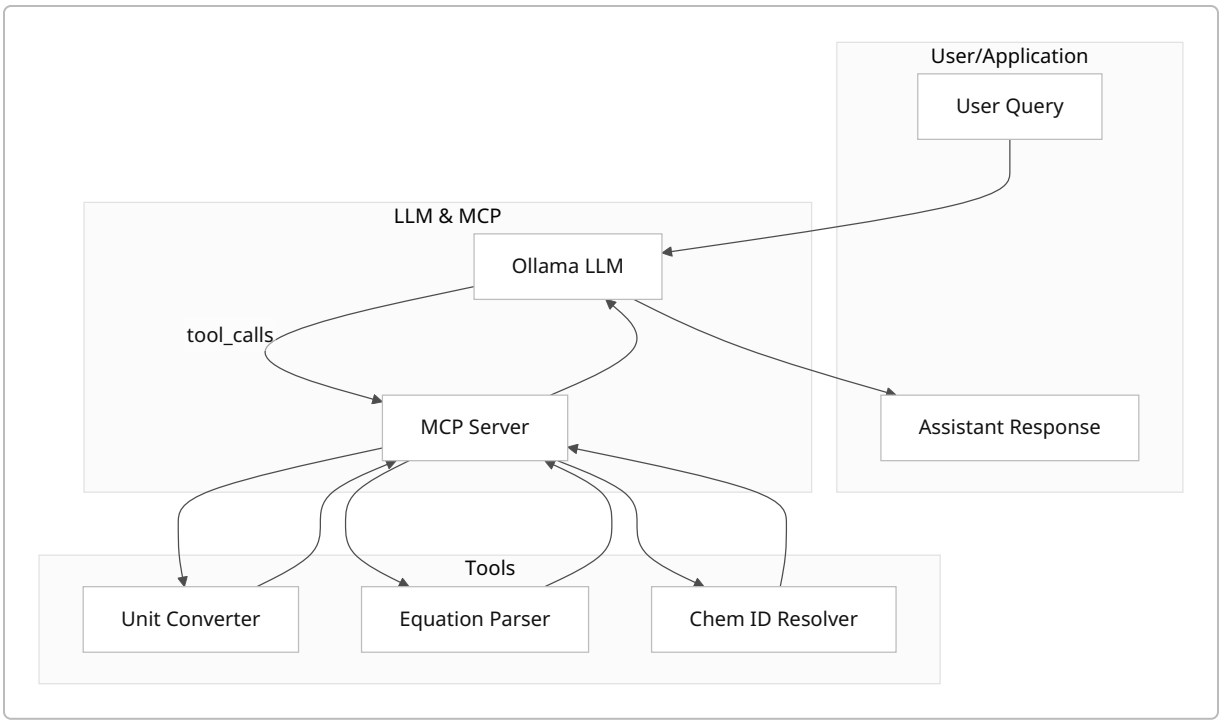
Plugin/Tool APIs: Ollama supports **function calling** via a "tools" list in chat requests. Each tool is defined by a JSON schema of its arguments (similar to OpenAI's function calling). For example, a tool definition might look like:

```
{
  "type": "function",
  "function": {
    "name": "get_temperature",
    "description": "Get current temperature",
    "parameters": {
      "type": "object",
      "required": ["city"],
      "properties": {
        "city": {"type": "string", "description": "City name"}
      }
    }
  }
}
```

The assistant can then emit a tool call with arguments, e.g. `{"tool_calls": [{"type":"function","function":{"name":"get_temperature","arguments":{"city":"New York"}}}]}` 【75†L104-L112】【75†L130-L138】. Ollama's SDKs (Python, JS) make this easier: you can pass Python functions or JS functions as tools directly【75†L175-L184】【75†L211-L220】. When the model calls a tool, the MCP server invokes the corresponding handler. Output flows back by appending a message role `tool` with the result. MCP servers support **parallel calls** too – you can request multiple tools in one turn and return all results before finalizing the response【75†L272-L281】. Ollama also supports *streaming* and *think/no_think* modes (Qwen-specific, see below), and accepts function metadata including descriptions and JSON Schema【75†L104-L112】.

Security & Isolation: By default, tools run on the host machine as part of the MCP server. For sensitive scenarios, one should sandbox them. Docker or microVM sandboxes can isolate the process and network (e.g. using Firecracker or gVisor). One approach is to run Ollama in a Docker container that has no internet access, with a separate "downloader" sidecar to fetch models【56†L238-L247】. Third-party services like HopX use hardware-level isolation so that each Ollama instance has its own kernel and filesystem 【55†L1782-L1786】. In general, best practices include least-privilege (run Ollama as non-root), network restrictions, and validating tool inputs. MCP servers can enforce schema validation on inputs, which helps prevent code injection attacks.

Data Flow: In a typical workflow, the user's query is sent to the Ollama chat endpoint (HTTP or SDK). The model may output a structured tool call (as JSON) instead of or in addition to text. The MCP server sees this call, executes the corresponding function/tool, and returns the result as a new message (role `tool`) back into the chat history. Ollama can then continue the conversation (e.g. finalize the answer) using the tool output. You can run through multiple iterations of tool calls if needed. The user sees a unified answer, unaware of the back-and-forth, unless verbose logging is enabled.



Model Hosting & Formats: Ollama supports many open-weight LLMs. It has built-in "Ollama format" (Safetensors or GGUF). The Importing Model docs show that you can import HuggingFace models and adapters as Safetensors or GGUF【77†L146-L154】. Supported architectures include Llama-2/3, Mistral-1/2, Mixtral, Gemma, and Phi-3【77†L146-L154】. Ollama can quantize models (q4_K_S, q4_K_M, q8_0 etc) during creation【77†L219-L227】. For example, `ollama create --quantize q4_K_S modelname` produces a 4-bit K-means quantized model. In practice, you can run *Safetensors* or *GGUF* models. Official docs explicitly list Safetensors and GGUF as supported inputs【77†L146-L154】【77†L219-L227】. Notably, Ollama uses GGUF (the newer open format by llama.cpp) and can convert adapters to GGUF. Popular models (Llama3, Mixtral, CodeLlama, Phi-3, etc.) are available in Ollama Cloud or can be loaded locally. This wide support means you can host qwen3:8b or other models in Ollama with minimal fuss.

Examples & Limitations: Ollama's CLI and SDK examples (above) show tool calling in action. Community guides demonstrate Ollama under Docker or integrated into agent frameworks. Limitations include that Ollama is relatively new, so some edge-case bugs may exist. It currently focuses on English-centric UIs but supports multilingual models. Large models (tens of billions of parameters) will need substantial RAM/VRAM. Real-time constraints may favor quantized 8-bit models for responsiveness. The MCP server itself should be kept lightweight to avoid becoming a bottleneck; rawveg's implementation is quite efficient (zero Node deps, lean TS). One limitation of the current function-calling interface is that, unlike OpenAI, Ollama's chat API still returns the final answer in one shot (i.e. no official streamed tool completion yet), though the underlying mechanism supports streaming.

2. Scientific Notation Standards

Scientific data uses many domain-specific formats. Here we review conventions and machine-readable standards:

- **SI Units & E-Notation:** The **International System of Units (SI)** is the official modern metric system 【82†L73-L80】. It defines base units (meter, kilogram, etc.) and prefixes (kilo-, milli-, etc.). A physical quantity is expressed as "number × unit". In many scientific contexts, numbers use **scientific (E) notation**: e.g. `1.23e-4` for 1.23×10^{-4}. This is universally understood in computing (IEEE floats often print this way). SI itself prefers using "×10^n" in printed text, but electronic data often uses `E` for ease. For example, "$3.50×10^5$ m" is equivalent to `3.50e5 m`. Significant figures and uncertainty are critical: measurement uncertainty should typically be rounded to 1–2 significant digits【84†L133-L137】. Reporting "$1.234(12)\times10^5$" indicates $1.234×10^5±0.012×10^5$. Tool input schemas can represent uncertainties explicitly (e.g. value plus plus/minus fields). Libraries like **Pint** (Python) handle SI unit parsing and conversion programmatically【47†L12-L14】. For example, Pint understands `3.5 km` and `3500 m` as equal. Data schemas (JSON) can include a `"unit"` field and a numeric `"value"`, or embed units in a string.

- **LaTeX / Math:** Scientific documents often use LaTeX for formulas. E.g. `$E=mc^2$` or `$3.5\\times 10^5$`. To integrate with an LLM, one could convert LaTeX to plain text or parse it. Tools like `sympy` can parse LaTeX math into symbolic form. For machine use, **MathML** or TeX-to-plaintext converters can be used. In writing, LaTeX's exponent syntax (`10^` or `\\times`) is standard, but JSON schemas might prefer numeric fields. A conversion tool can take a string like `"$2.5 \\times 10^{-3}$"` and output `0.0025` or vice versa.

- **Chemistry (IUPAC, InChI, SMILES):** Chemical compounds have systematic names (IUPAC), but those are verbose. Two compact identifiers are **InChI** and **SMILES**. InChI (International Chemical Identifier) is an IUPAC standard that encodes a molecule's structure into a layered string【40†L185-L193】. It's non-proprietary and widely used in data archives. For example, acetic acid has InChI `InChI=1S/C2H4O2/c1-2(3)4/h1H3,(H,3,4)`. **SMILES** (Simplified Molecular Input Line Entry System) is another standard by Daylight/ACS, using ASCII strings to describe structure【43†L185-L193】. E.g. `CC(=O)O` is acetic acid. OpenSMILES is an open specification. Both InChI and SMILES omit spaces and use characters for bonds. Tools like RDKit can convert between InChI/SMILES and names. In JSON, one might store `{ "inchi": "...", "smiles": "..." }`. IUPAC names can be parsed by ChemAxon or OPSIN, but are more variable.

- **Biological Sequences:** DNA/RNA/proteins use formats like **FASTA**. A FASTA record starts with a `>id` line, then sequence (e.g. `>seq1\nACTGGTC...\n`). Submissions often wrap lines at ~80 chars 【45†L121-L130】. Other formats include **GenBank** or **FASTQ** (with quality scores). For machine use, FASTA is simple: a JSON representation could be `{"id": "seq1", "sequence": "ACTG..."}`. Ontologies exist (e.g. Sequence Ontology for features). Tools should ensure sequences use valid characters (A,C,T,G/U for nucleotides; 20 amino acid letters for proteins).

- **Units Libraries & Data Standards:** Many libraries encode these standards. Besides Pint, there is **Unidata UDUNITS** (used in netCDF data) and **Astropy Units** (for astronomy). For domain-specific structured data, ontologies like the **OBO Foundry** provide standardized biological terms【49†L76-

L78】. The **UMLS** is a biomedical metathesaurus linking thousands of terms and codes【51†L88-L92】. For example, drug units or medical terms come from UMLS. JSON schemas for scientific data often follow projects like Schema.org's QuantitativeValue or community schemas (OpenAPI specs for bioinformatics tools).

- **Edge Cases:** Handling **significant figures** and **uncertainty** is tricky. By convention, uncertainties are reported with 1–2 sig figs and aligned with the value (e.g. `1.234±0.056`). Values like "1.230" imply precision. Certain units (like ppm) have SI guidelines: SI discourages "ppb/ppm" for exactness and prefers "10^{-6}" forms【79†L49-L57】. Unit conversion must handle compound units (e.g. m/s^2) and non-metric (inch, lb) when needed. A robust tool would use a vetted conversion database. JSON schemas can encode units (like `schema.org` units or custom "value"+"unit" fields). Ontologies (OBO, UMLS) provide machine-readable term IDs (e.g. CHEBI for chemicals, GO for gene products).

3. Qwen3-8B Model

Tokenization & Vocabulary: Qwen3 uses a **byte-level BPE** tokenizer with a very large vocabulary (≈151,646 tokens)【65†L138-L146】. It was trained to have *no unknown tokens* – every text can be tokenized exactly. Typical English uses ~3–4 chars/token, Chinese ~1.5–1.8 chars/token on average【65†L149-L152】. There are special "control tokens" for formatting turns: e.g. `<|endoftext|>`, `<|im_start|>`, `<|im_end|>` marking end-of-sequence or dialogue turn boundaries【65†L169-L177】. The tokenizer has no fixed pad token by default (models use attention masks), and uses BPE so even new words break into subword pieces. In practice, you use Hugging Face's `AutoTokenizer.from_pretrained("Qwen/Qwen3-8B")` and then call `.tokenize(...)` or `.apply_chat_template(...)` for chat formatting【58†L418-L427】【71†L215-L224】.

Context Window: Out-of-the-box, Qwen3-8B supports ~32K tokens of context【58†L526-L534】. Unsloth's documentation notes a **native 128K context** variant (via RoPE/YaRN scaling)【61†L137-L144】. In either case, this is far larger than many LLMs. In Hugging Face code, one sets `max_new_tokens=32768` as needed【58†L418-L427】. Practically, using such large context may require special inference engines (vLLM, SGLang) or Qwen's prefix-scaling support. But for scientific QA, even 8–16K may suffice. Qwen also supports a "thinking mode" which influences context handling (see below).

Fine-tuning & Adaptation: As an 8B model, Qwen3-8B can be fine-tuned with common techniques. Hugging Face PEFT/LoRA adapters work (e.g. Qwen-Agent or the Unsloth notebooks do LoRA on 4-bit quantized weights【61†L151-L159】). Unsloth reports being able to fine-tune Qwen3 (14B) in 4-bit on moderate hardware【61†L137-L144】. In general, one could use Hugging Face's `transformers` + `peft` libraries. Alternately, Qwen has its own *Dynamic 2.0* format and supports offline adapters. Quantization can be applied post-hoc (as shown in Ollama's docs) so a fine-tuning pipeline might combine 4-bit weights with low-rank updates. For on-device small deployments, methods like **LoRA/QLoRA** are recommended. Qwen does not (to our knowledge) support true zero-shot "tool-use API" like OpenAI's ChatGPT Plugins out-of-the-box, but one uses the same function-calling mechanism as above.

Tool-use APIs & Prompting: Qwen3 provides built-in support for function-calling prompts. Its documentation advises using **Hermes-style** templates for function/tool invocation【69†L123-L131】. For example, in code you define an available tool list (as JSON schema or Python/JS functions) and then set `think=True` to allow the model to emit tool calls. The Qwen docs explicitly suggest using Qwen-Agent

(their agent framework) or enabling a "reasoning parser" in vLLM/sglang for processing `<think>...</think>` blocks【71†L280-L284】. In practice, an example in their blog shows:

```
from transformers import AutoTokenizer, AutoModelForCausalLM
tokenizer = AutoTokenizer.from_pretrained("Qwen/Qwen3-8B")
model = AutoModelForCausalLM.from_pretrained("Qwen/Qwen3-8B")
text = tokenizer.apply_chat_template(messages, tokenize=False,
add_generation_prompt=True, enable_thinking=True)
outputs = model.generate(**tokenizer(text, return_tensors="pt"),
max_new_tokens=32768)
```

Here `enable_thinking=True` turns on chain-of-thought mode by default【71†L219-L227】. Within the chat, the user can even include `/think` or `/no_think` tokens to switch modes turn-by-turn: if `/no_think` is seen, the model's output will have no `<think>...</think>` content【71†L280-L284】. This is Qwen's hybrid solution for fast vs. deep reasoning. The assistant's output will be wrapped like `<think>internal reasoning</think><answer>final answer</answer>`, which one can parse. The Hugging Face example shows parsing out the indices of `</think>` 【71†L236-L245】. If using the Ollama API, the same effect is achieved by setting `think: true` and including `/think` tags in the system/user messages. After a tool call is used, one adds the result as a message role `tool` and calls `ollama.chat` again, akin to a typical function-calling loop【75†L175-L184】.

Constraints & Patterns: As an 8B model, Qwen3-8B has strong language and reasoning ability, but one should craft prompts carefully. For example, always include a system or user instruction clarifying that the model should use tools for exact computations (e.g. "Use the provided unit-conversion tool for calculations."). Use JSON schemas to constrain arguments. Qwen's own examples show defining `name` / `description` and parameter `type:object` with required fields【75†L104-L112】. Because small models can hallucinate, a strategy is to allow or encourage function calls early: e.g. prompt "You may use the calculator tool for any arithmetic." The "thinking mode" is optional; for concise numeric tasks you may disable it (set `enable_thinking=False`) so the model jumps straight to the answer. Always monitor token usage since 32K context is generous but finite. Keep chain-of-thought tokens to a minimum if focus is on correctness.

4. Integration Design

Our goal: let *qwen3:8b via Ollama MCP* handle scientific queries by offloading notation-heavy work to custom tools. We outline a design pattern:

- **Adapter Pattern:** Implement each scientific helper as an MCP "tool" with a JSON I/O contract. For example, a unit-conversion tool might have schema:

```
{
  "name": "unit_convert",
  "description": "Convert a numeric value from one unit to another",
  "parameters": {
    "type": "object",
```

```
    "properties": {
      "value": {"type": "number", "description": "Amount to convert"},
      "from_unit": {"type": "string"},
      "to_unit": {"type": "string"}
    },
    "required": ["value","from_unit","to_unit"]
  }
}
```

This mirrors Ollama's expected format【75†L104-L112】. In code (TypeScript with rawveg/ollama-mcp), it looks like:

```
export const toolDefinition = {
  name: "unit_convert",
  description: "Convert numeric units",
  inputSchema: {
    type: "object",
    properties: {
      value: { type: "number" },
      from_unit: { type: "string" },
      to_unit: { type: "string" }
    },
    required: ["value","from_unit","to_unit"]
  }
};
export async function handler(params) {
  // parse units and perform conversion...
}
```

The handler would use a units library (like Pint, see below). The JSON schema enforces correct types before calling the handler, adding security. Any schema validation error can return a clear error message to the user.

- **Data Schemas and API Contracts:** Define a shared data interchange format. For unit conversion, it's (number, from, to) → number. For a "scientific notation parser", it might be text→ number. For LaTeX↔plain, strings in/out. For chemicals: input could be an identifier (name, InChI, SMILES) and output could be a structured JSON with different representations. For biological sequences: input raw sequence (string) or FASTA chunk, output JSON of formatted lines or features. We recommend using JSON Schema to define each API (tools list above). This ensures the LLM knows exactly what keys to supply. In effect, the MCP server's tool-definition is the API spec.

- **Example Tool Implementations:**

- *Unit Conversion Tool:* Use Pint (Python) or an equivalent. Pint supports parsing and converting SI and US units【47†L12-L14】. A handler could do: `qty = ureg.Quantity(value, from_unit);`

`result = qty.to(to_unit)`; return `{"result": result.magnitude}`. Ensure dimension check and handle exceptions (e.g. incompatible units).

- *Scientific Notation Parser:* Accept a string or JSON (e.g. "1.23×10^4" or `{"mantissa":1.23,"exp":4}`) and output a numeric value. A quick regex or use Python's `decimal` library. Edge: allow both `E` and unicode "×10^" forms.
- *LaTeX⇄Plain Converter:* Use a library (e.g. `sympy.parsing.latex.parse_latex` or `latex2text`). Example: convert LaTeX math to plaintext or evaluate expression. For instance, input `"$3.2\\times10^{-3}$"`, output `0.0032`.
- *Chemical ID Resolver:* Using RDKit (Python), take an input (could be name, InChI or SMILES) and return JSON with `{inchi: "...", smiles: "...", name: "Acetylsalicylic acid", formula: "C9H8O4"}`. RDKit can generate InChI/SMILES from one another. If RDKit not available, a web API (PubChem REST) could be used as a fallback.
- *Sequence Formatter:* Using Biopython, take a raw sequence string and output FASTA format lines of 60 chars. Or verify a FASTA record and return JSON of id and sequence. For example, `">id\nACTG..."`.

Each handler should catch and report errors. For example, if a chemical name isn't found, return `{"error":"Unknown compound"}` which the MCP server will display as tool output. Always return JSON-serializable types (strings, numbers, booleans, objects, arrays).

- **Error Handling:** Within each tool, validate inputs (JSON Schema enforces basic types). Wrap logic in try/catch. If an error occurs (e.g. invalid unit, parse failure), return a structured error (e.g. raise an exception or return an object with `error` field). The MCP server should intercept unhandled exceptions and send an appropriate message back. In practice, one might catch exceptions and do `return { error: e.message }`. On the Ollama side, this appears as the tool's content, which the model can include or re-ask. Always prefer clear error messages like "Unrecognized unit" rather than a stack trace.

- **Testing Strategies:** Write **unit tests** for each tool: valid cases, boundary cases, and invalid input. For example, unit tests using `pytest` or Jest. Test conversions (e.g. 100 cm→1 m, etc.), LaTeX parsing (simple and complex math), chemical lookups. Integration tests: simulate a full chat with known queries, verifying the final answer. One can script the Ollama CLI or use the Python SDK to send a message and check the JSON response. Monitor logs for unexpected tool calls. Continuous integration pipelines should include these tests.

- **Evaluation Metrics:** For correctness of answers, use task-specific metrics. For numeric tasks, one could use **relative error** or compare to a tolerance (e.g. ±0.5% for physical constants). For formatted outputs (SMILES/InChI), use exact string match (these are precise). For LaTeX conversion, compare to a math expression parser's output. Overall accuracy could be measured by test sets (e.g. known unit conversion problems, sequence formatting tasks) and compute success rate. In an engineering setting, one might track "fraction of questions answered correctly" on a benchmark. More formal metrics (BLEU, ROUGE) are less applicable since we want exact factual output. For chemical name resolution, one could use database hits as ground truth. Logging and manual review will also be important.

- **Security & Performance:** Because the tools may handle sensitive data (scientific or personal), run the MCP server on a secure host. Avoid network calls in tools unless absolutely needed; if calling

external APIs (e.g. PubChem), ensure SSL and timeouts. Use containers or virtual environments with minimal privileges. Monitor CPU/memory: some conversions (especially chemistry) can be CPU-intensive (e.g. RDKit), so consider caching frequent results. Ollama's model inference is GPU/CPU-bound; tools run on the CPU of the host. We recommend using low-latency libraries: Pint and regex are fast, Biopython is lightweight. For large workloads, consider offloading heavier tasks to asynchronous workers (MCP can handle streaming, but rawveg is currently synchronous). For deployment, containerize the MCP server (Docker) and pin dependencies. Also ensure the Ollama process is monitored and restarted if it dies.

```
flowchart LR
   subgraph Ollama MCP Integration
     Q[User Query (chat)] --> A[Ollama Model (qwen3:8b)];
     A --> B{Tool Needed?};
     B -- yes --> C[JSON tool_call (e.g. unit_convert)];
     C --> D[MCP Server (validate schema)];
     D --> E1[UnitConverter], E2[EquationParser], E3[ChemResolver],
E4[SeqFormatter];
     E1 -- result --> D;
     E2 -- result --> D;
     E3 -- result --> D;
     E4 -- result --> D;
     D --> F[Return result message to LLM];
     F --> A;
     B -- no --> A;
     A --> G[Final answer text];
     G --> Q;
   end
```

5. Implementation Plan

We break the integration into phases. Each phase has milestones, effort (estimated Low/Med/High), and potential risks:

Step / Milestone	Effort	Risk	Notes
1. Environment Setup: Install Ollama (CLI/SDK), MCP server framework (e.g. rawveg or Python). Obtain qwen3:8b model. Test simple chat.	Low	Low	Standard install, follow Ollama docs.
2. Define Tools & Schemas: Write JSON schemas for each tool (unit, parser, LaTeX, chemical, sequence). Create stub handlers.	Medium	Low	Requires design of input/output contract. Risk of missing edge cases.
3. Implement Tools: Code each handler (use Pint, regex, RDKit, Biopython, etc). Validate with unit tests.	High	Med	Moderate effort for chemistry/latex. Potential libs issues.

Step / Milestone	Effort	Risk	Notes
4. Integrate into MCP: Plug tools into MCP server (TypeScript files or Python modules). Ensure server loads them. Test calls with Ollama chat.	Medium	Low	Minor: mostly wiring. Key is schema alignment.
5. Prompt Engineering: Craft system/user prompts to trigger tools appropriately. Fine-tune the "chat template". Experiment with /think mode.	Medium	Med	Model may misinterpret prompts; iterative tuning needed.
6. Testing & QA: Run end-to-end tests: feed scientific queries, check answers. Validate handling of edge cases (invalid input, large values).	High	Med	Need broad test coverage. Risk: silent errors.
7. Evaluation Metrics: Develop evaluation dataset (e.g. unit conversion problems, formula parsing examples). Compute accuracy, error tolerance.	Medium	Low	Low risk; mostly data collection.
8. Security Hardening: Containerize MCP, restrict network, audit dependencies. Pen-test inputs for injection.	Medium	Med	Ensuring no code injection in tool calls.
9. Deployment & Monitoring: Deploy on target (on-prem or cloud sandbox). Set up logging/alerts. Monitor performance.	Medium	Med	Performance tuning (multi-threading?).

Each "High" effort step may take weeks, others days. Key risks include model hallucination (mitigated by schema constraints), library bugs (test coverage), and integration bugs. However, each component is modular (lower coupling): tools can be developed/tested independently.

6. Comparison Tables & Diagrams

Integration Methods: Below compares approaches for tool integration:

Method	Ease of Integration	Tool Definition	Isolation	Trade-offs
Ollama MCP (e.g. rawveg)	High	JSON schema via `toolDefinition`【31†L586-L591】【75†L104-L112】	Medium	Easiest use of Ollama features; limited to Ollama-compatible formats.
OpenAI-style Function Calls	Medium	Similar JSON schema	Low (no sandbox)	Works if using an OpenAI-like API, but local model must mimic it.

Method	Ease of Integration	Tool Definition	Isolation	Trade-offs
In-Model Prompting Only	Low	N/A	N/A	No formal API; model may hallucinate syntax.
External Microservices	High	Custom REST APIs	High	More work to set up; can use any language; requires secure comms.

Scientific Formats:

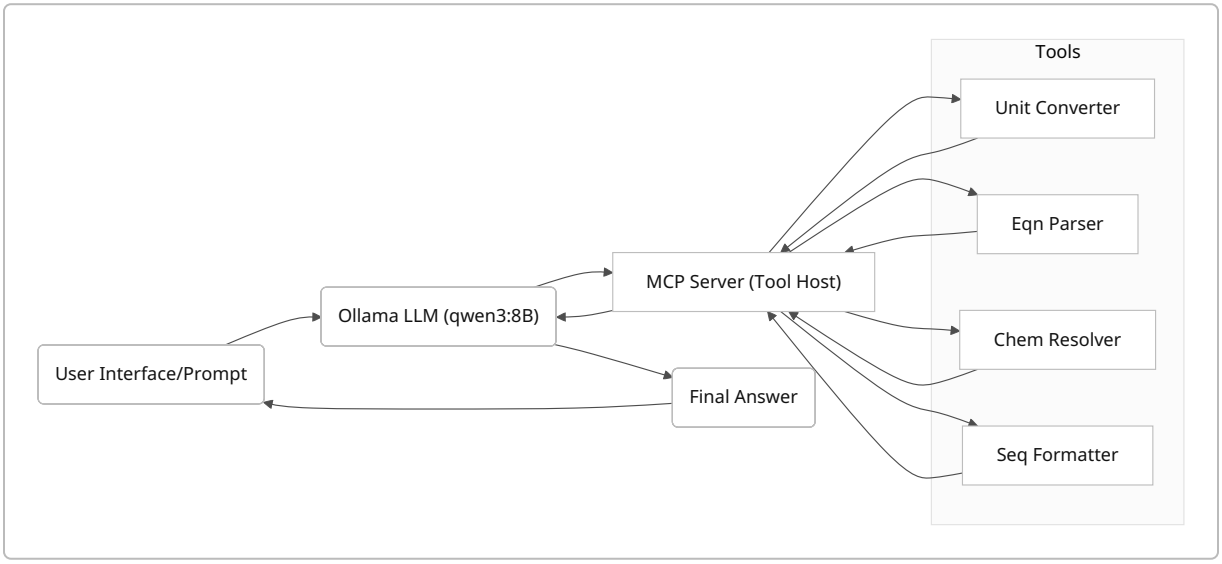
Notation	Example	Machine Format	Use Case	Libraries/Specs
SI (units)	`9.81 m/s^2`	JSON: {value, unit} or numeric with unit string	Physical quantities	Pint【47†L12-L14】, UDUNITS
E-Notation	`6.02e23`	JSON number or scientific string	Chemical/ constants	IEEE floats, None needed
LaTeX Math	`$E=mc^2$`	MathML or plaintext	Equations in text	Sympy, latex2text
IUPAC Name	“2,4-Dimethylpentane”	Plain text/name	Chemical naming	IUPAC rulebooks
SMILES	`CC(=O)O`	String	Cheminformatics	OpenSMILES spec, RDKit
InChI	`InChI=1S/`...	String	Database indexing	IUPAC InChI docs【40†L185-L193】
FASTA Seq	`>id\nACGT...`	JSON	Genomic/protein data	NCBI FASTA spec【45†L121-L130】
Ontology ID	`CHEBI:15377`	String	Concept coding (chem)	OBO Foundry【49†L76-L78】, UMLS【51†L88-L92】

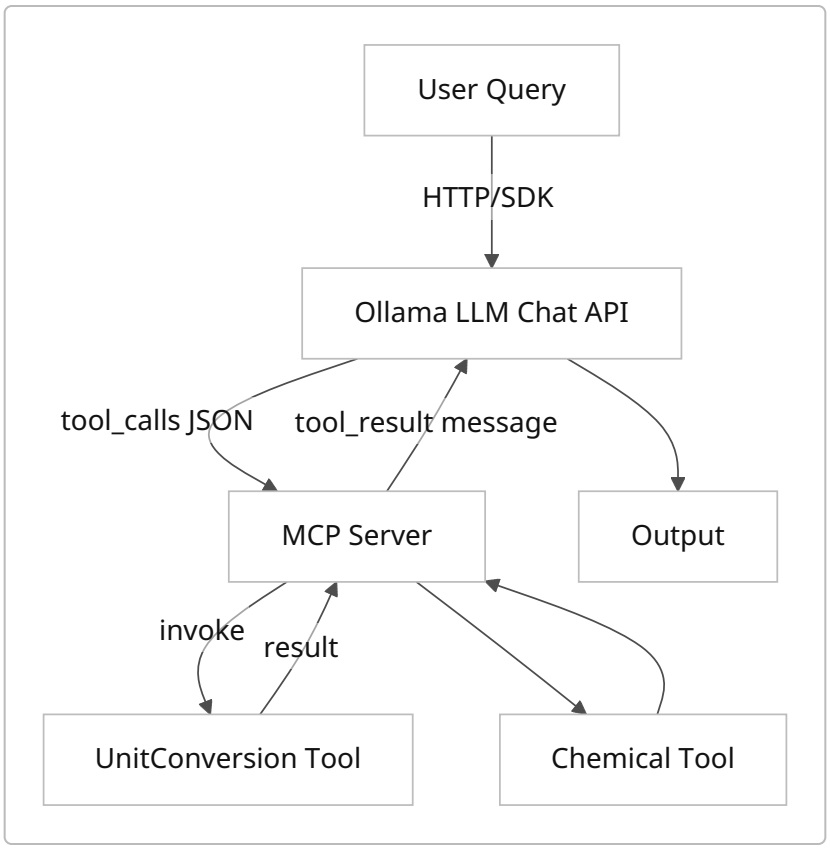
Libraries & Tools:

Task	Library/Tool	Language	Notes/Trade-offs
Units/Conversion	Pint【47†L12-L14】, Astropy.units	Python	Pint is easy, supports SI/imperial; Astropy for astronomy units.

Task	Library/Tool	Language	Notes/Trade-offs
LaTeX Parsing	Sympy, latex2mathml	Python	Sympy can parse math; latex2text for general latex→text.
Chemistry (convert)	RDKit, OpenBabel	Python/C	RDKit has Python API for SMILES/InChI; requires C++ build.
Sequence Formats	Biopython SeqIO	Python	Parses/writes FASTA/GenBank.
JSON Schema Validate	AJV (JS), `jsonschema` (Py)	JS/Py	Validate tool inputs per schema.
Embedding Notation	MathJax, KaTeX	JS	Render LaTeX in UIs if needed.

Mermaid Diagrams: The above diagrams illustrate (1) high-level architecture showing Ollama, MCP server, and tools, and (2) detailed dataflow of a chat with tool calls.





Recommended Sources: We relied on Ollama's official docs for architecture and tool calling【21†L96-L104】【75†L104-L112】, on GitHub examples (rawveg/ollama-mcp) for server design【31†L586-L591】, on Ollama's import docs for supported model formats【77†L146-L154】【77†L219-L227】, on Qwen's docs for tokenization and prompting【65†L138-L146】【71†L280-L284】, and on standards sites for notations (NIST for SI【82†L73-L80】, IUPAC for InChI【40†L185-L193】, Wikipedia for SMILES【43†L185-L193】, NCBI for FASTA【45†L121-L130】, and UMLS/OBO sites for ontologies【49†L76-L78】【51†L88-L92】). These were prioritized to ensure authoritative coverage. Other community guides (e.g. Unsloth, HopX, Qwen-Agent) informed best practices.
